



#### INTEGRATION INFORMATION REQUEST

# LSN Engineering Console

## Physical Firmware Integration

### ESP32 / WT32-ETH01

A concise handoff explaining the Console's current status, the firmware information needed, and how physical hardware testing will be enabled.

|                           |                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------|
| <b>CURRENT<br/>STATUS</b> | <b>Simulation is available.</b><br><b>Physical network communication is not yet enabled.</b> |
|---------------------------|----------------------------------------------------------------------------------------------|

Prepared for firmware engineering review  
Saber Industrial Applications

# Purpose and current status

The LSN Engineering Console is prepared for integration with the physical ESP32/WT32-ETH01 firmware, but **physical device communication is not enabled in the current version**.

Hardware Mode is intentionally locked because the ESP32's final network and communication interface has not yet been supplied. The Console does not currently send discovery packets, open a device session, or issue commands to physical hardware.

## Why this is intentionally disabled

We do not want to invent a protocol or require unnecessary firmware changes. The preferred approach is to understand the interface already implemented—or planned—by the firmware engineer, then adapt the Console to match it while preserving the required safety behavior.

## What the Console already provides

- A working Simulation Mode for reviewing and validating expected LSN behavior.
- Device identity, status, control, diagnostics, runtime, fault, and guided test screens.
- A configurable Device Profile for firmware fields and protocol mappings.
- Safety gates for enable/disable behavior and connection-loss handling.
- Telemetry freshness rules that distinguish live, stale, and unknown evidence.
- Firmware integration package generation and implementation checklists.
- A packaged Windows architecture designed to keep future network sockets in the isolated Electron main process.

## Important validation boundary

Simulation results help verify the logical contract, but they are not physical firmware validation. Only evidence captured through the completed hardware connection may be treated as physical test evidence.

## Requested approach

Please share the interface that currently exists—or the interface you intend to implement. Incomplete or provisional details are acceptable; unresolved items can remain marked **TBD**. After review, the Console will be adapted to that real interface wherever practical.

# Information requested

Please provide whatever is currently known in each area. Existing source headers, packet captures, test scripts, or short examples are welcome and may answer several sections at once.

**1**

## Physical network

- Ethernet or Wi-Fi.
- DHCP or static addressing.
- Default IPv4 address or hostname, if applicable.

**2**

## Device discovery

- How should the application locate the ESP32?
- Fixed IP, mDNS, UDP broadcast, EtherNet/IP ListIdentity, or another method?

**3**

## Communication protocol

- EtherNet/IP/CIP, Modbus TCP, HTTP, WebSocket, custom TCP/UDP, or another protocol.
- TCP/UDP port numbers and whether a session must be established first.

**4**

## Device identity

- Product name, hardware model, firmware version, and serial number.
- Vendor ID, device type, and product code, if applicable.

**5**

## Commands and telemetry

- How values are read and how enable/disable commands are sent.
- Mappings for Ready, fault, output-active, timer, runtime, and enable count.
- Command IDs, object paths, registers, endpoints, or message formats.

# Information requested — continued

6

## Data representation

- Data types and field sizes.
- Byte order, string encoding, and bit/byte packing.
- Enum values and fault-code definitions.

7

## Safety behavior

- Expected behavior when communication is lost.
- Firmware watchdog availability and timeout behavior.
- Conditions that must be satisfied before enable is accepted.
- How firmware confirms that the physical output is active.

8

## Useful examples

- Existing test script, application, or command-line tool.
- Firmware header containing protocol definitions.
- Example request/response packets or a successful packet capture.
- Any existing interface or bring-up documentation.

# What happens after we receive this information

1

### Review

Confirm the actual discovery, transport, data mapping, and safety contract.

2

### Integrate

Add device discovery or manual IP connection, identity verification, and session handling.

3

### Exercise

Enable live reads, safe commands, fault handling, and physical guided tests.

4

### Validate

Run the acceptance checklist on a real WT32-ETH01 and keep evidence separate from simulation.

### Shared objective

Make the Console conform to the firmware implementation wherever practical—not force a new protocol onto the ESP32 without reviewing what already exists. Once integrated, the Windows Console will provide the physical connection and test workflow needed to verify the firmware against the LSN contract.